

AI Crop Doctor - YOLO Model Training and Testing

Dataset Breakdown:

- **Total Number of Images:** 22,736 images.
- **Total Number of Classes (Labels):** 24 distinct classes.
- **Total File Size:** 2.67 GB.

 What is inside the 24 Labels?

Crop Name	Number of Disease/Healthy States	Approx. Image Count
Tomato	10 Classes (Blight, Mold, Virus, Healthy, etc.)	~9,500
Apple	4 Classes (Scab, Black Rot, Rust, Healthy)	~3,800
Corn (Maize)	4 Classes (Rust, Blight, Gray Spot, Healthy)	~3,700
Grape	4 Classes (Black Rot, Esca, Blight, Healthy)	~3,600
Potato	2 Classes (Early/Late Blight, Healthy)	~2,100
Total	24 Total Labels	22,736 Total

Step 1: Install Required Libraries

Copy and paste this exact command into your VS Code terminal and hit Enter. This will install YOLO, the dataset handlers, and image processing tools.

PowerShell

Create evn.

`pip install ultralytics`

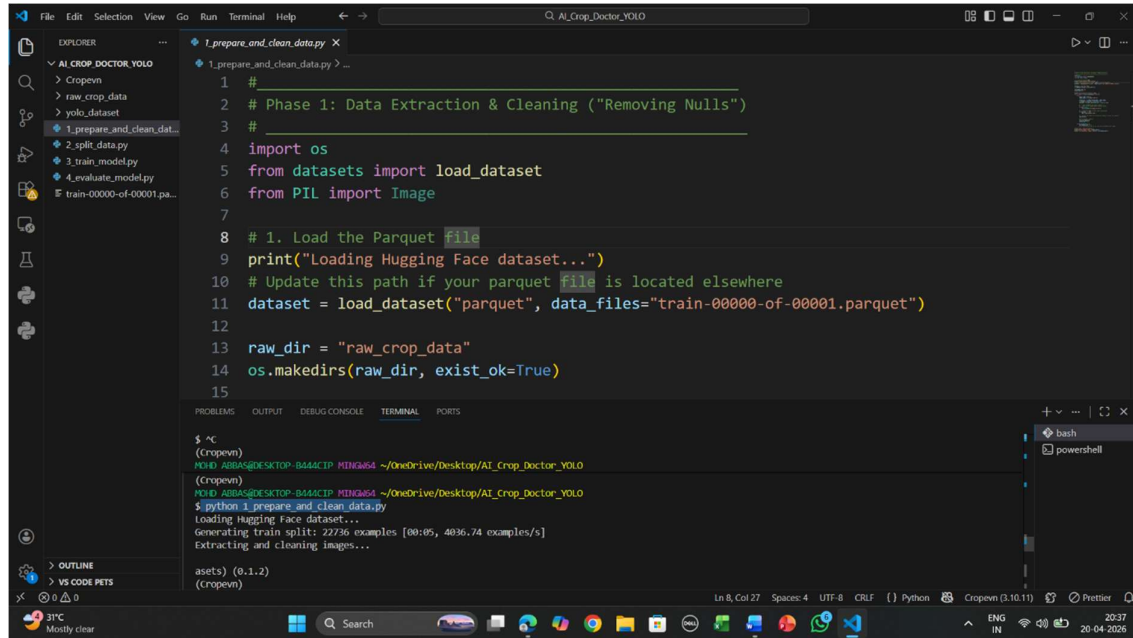
`pip install datasets pillow split-folders`

Run file by file

1. `python 1_prepare_and_clean_data.py`
2. `python 2_split_data.py`
3. `python 3_train_model.py`
- 4.

Model Training Output:

Phase 1: Data Extraction & Cleaning ("Removing Nulls")



```

1 #
2 # Phase 1: Data Extraction & Cleaning ("Removing Nulls")
3 #
4 import os
5 from datasets import load_dataset
6 from PIL import Image
7
8 # 1. Load the Parquet file
9 print("Loading Hugging Face dataset...")
10 # Update this path if your parquet file is located elsewhere
11 dataset = load_dataset("parquet", data_files="train-00000-of-00001.parquet")
12
13 raw_dir = "raw_crop_data"
14 os.makedirs(raw_dir, exist_ok=True)
15

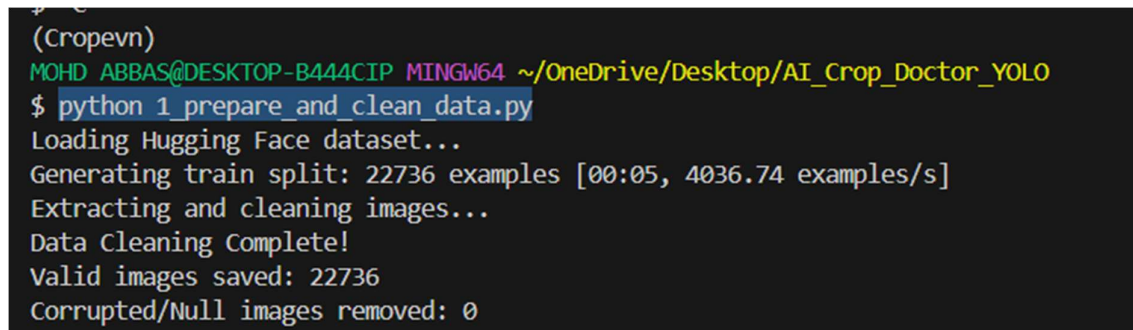
```

```

$ ^C
(Cropevn)
MOHD ABBAS@DESKTOP-B444CIP MINGW64 ~/OneDrive/Desktop/AI_Crop_Doctor_YOLO
$ python 1_prepare_and_clean_data.py
Loading Hugging Face dataset...
Generating train split: 22736 examples [00:05, 4036.74 examples/s]
Extracting and cleaning images...
assets) (0.1.2)
(Cropevn)

```

-----Or-----



```

(Cropevn)
MOHD ABBAS@DESKTOP-B444CIP MINGW64 ~/OneDrive/Desktop/AI_Crop_Doctor_YOLO
$ python 1_prepare_and_clean_data.py
Loading Hugging Face dataset...
Generating train split: 22736 examples [00:05, 4036.74 examples/s]
Extracting and cleaning images...
Data Cleaning Complete!
Valid images saved: 22736
Corrupted/Null images removed: 0

```

Mean:

That is a perfect output!

Seeing 0 corrupted/null images removed means the dataset you downloaded from Hugging Face is exceptionally clean and high-quality. You now have exactly 22,736 healthy crop images successfully extracted into your raw_crop_data folder.

Phase 2: Split Data for YOLO:

```
(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 2_split_data.py
Splitting data into Training and Validation sets...
Copying files: 22736 files [04:42, 80.35 files/s]
Splitting complete! Data is ready for YOLOv8 in the 'yolo_dataset' folder.
```

Mean:

Awesome, that output is spot on. Your 22,736 images are now perfectly distributed into train and val folders inside yolo_dataset.

For Image Classification (What you are doing now): YOLO only needs to know *what* the image is as a whole. Therefore, the folder name itself acts as the label.

In your screenshots, the folders are named 0, 1, 2, etc.

- If an image is inside folder 0, YOLO automatically labels it as class 0.

Phase 3: Train the YOLOv8-cls Model

```
(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 3_train_model.py
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n-cls.pt to 'yolov8n-cls.pt': 100% ————— 5.3MB 2.5MB/s 2.1s
Starting YOLOv8 training...
```

25 epochs for training:

If you look closely at the math in your screenshots, it breaks down perfectly:

- **Train Folder: 18,180 images**
- **Val Folder: 4,556 images**
- **Total = 22,736 images**

```
Transferred 156/158 items from pretrained weights
train: Fast image access (ping: 0.10.0 ms, read: 2.30.9 MB/s, size: 13.3 KB)
train: Scanning C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train... 18178 images, 2 corrupt: 100% ————— 18180/18180 1.1Kit/s 17.
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\21\img_19987.jpg: ignoring corrupt image/label: image size (8, 48) <10 pixels
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\23\img_22308.jpg: ignoring corrupt image/label: image size (1, 30) <10 pixels
train: New cache created: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train.cache
val: Fast image access (ping: 0.10.0 ms, read: 3.20.9 MB/s, size: 13.5 KB)
val: Scanning C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\val... 4556 images, 0 corrupt: 100% ————— 4556/4556 1.2Kit/s 3.9s
val: New cache created: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\val.cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...
1 -1 1 4672 ultralytics.nn.modules.conv.Conv [16, 32, 3, 2]
2 -1 1 7360 ultralytics.nn.modules.block.C2f [32, 32, 1, True]
3 -1 1 18560 ultralytics.nn.modules.conv.Conv [32, 64, 3, 2]
4 -1 2 49664 ultralytics.nn.modules.block.C2f [64, 64, 2, True]
5 -1 1 73984 ultralytics.nn.modules.conv.Conv [64, 128, 3, 2]
6 -1 2 197632 ultralytics.nn.modules.block.C2f [128, 128, 2, True]
7 -1 1 295424 ultralytics.nn.modules.conv.Conv [128, 256, 3, 2]
8 -1 1 460288 ultralytics.nn.modules.block.C2f [256, 256, 1, True]
9 -1 1 360984 ultralytics.nn.modules.head.Classify [256, 24]
YOLOv8n-cls summary: 56 layers, 1,469,032 parameters, 1,469,032 gradients, 3.4 GFLOPs
Transferred 156/158 items from pretrained weights
train: Fast image access (ping: 0.10.0 ms, read: 2.30.9 MB/s, size: 13.3 KB)
train: Scanning C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train... 18178 images, 2 corrupt: 100% ————— 18180/18180 1.1Kit/s 17.
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\21\img_19987.jpg: ignoring corrupt image/label: image size (8, 48) <10 pixels
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\23\img_22308.jpg: ignoring corrupt image/label: image size (1, 30) <10 pixels
train: New cache created: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train.cache
4 -1 2 49664 ultralytics.nn.modules.block.C2f [64, 64, 2, True]
5 -1 1 73984 ultralytics.nn.modules.conv.Conv [64, 128, 3, 2]
6 -1 2 197632 ultralytics.nn.modules.block.C2f [128, 128, 2, True]
7 -1 1 295424 ultralytics.nn.modules.conv.Conv [128, 256, 3, 2]
8 -1 1 460288 ultralytics.nn.modules.block.C2f [256, 256, 1, True]
9 -1 1 360984 ultralytics.nn.modules.head.Classify [256, 24]
YOLOv8n-cls summary: 56 layers, 1,469,032 parameters, 1,469,032 gradients, 3.4 GFLOPs
Transferred 156/158 items from pretrained weights
train: Fast image access (ping: 0.10.0 ms, read: 2.30.9 MB/s, size: 13.3 KB)
train: Scanning C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train... 18178 images, 2 corrupt: 100% ————— 18180/18180 1.1Kit/s 17.
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\21\img_19987.jpg: ignoring corrupt image/label: image size (8, 48) <10 pixels
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train\23\img_22308.jpg: ignoring corrupt image/label: image size (1, 30) <10 pixels
Ln 21, Col 6 Spaces: 4 UTF-8 CRLF ( ) Python 88 Cropevn (3.10.11) 87
```

What is an "Epoch"?

An Epoch is one complete pass through your entire training dataset.

Since your training folder has about 18,178 images, 1 Epoch means the YOLO model has looked at every single one of those 18,178 images exactly once, made its guesses, and updated its internal math to get better.

2. What is "top1_acc"?

top1_acc stands for **Top-1 Accuracy**.

When YOLO looks at a leaf image, it calculates the probability for all 24 possible diseases and ranks them.

Important Terms for ecochs output analysis:

Term in Log	What it means in your specific project
Instances (value: 2)	The leftover images in the final batch. You have 18,180 training images and set a batch=16. If you divide 18,180 by 16, you get 1136 full batches, with exactly 2 images left over. This column is just reporting the number of images processed in that very last fractional step of the epoch.
top5_acc (value: 100% or 1)	Top-5 Accuracy. This means that when the model evaluates a leaf, the actual correct disease is present somewhere within its top 5 highest-confidence guesses 100% of the time. (Your top1_acc tells you how often it is the absolute #1 guess).
1137/1137	Training Progress Bar. This shows how many "batches" of images the model has processed for that epoch. Since $18,180 \text{ images} \div 16 \text{ images per batch} = 1136.25 \text{ batches}$, it takes exactly 1137 steps to complete one full pass of your training data.
143/143	Validation Progress Bar. YOLO is smart and automatically doubles the batch size during the validation testing phase (from 16 to 32) to save time, because it doesn't need to consume memory for learning here. Your $4,556 \text{ validation images} \div 32 \text{ images per batch} = 142.37$, which rounds up to 143 steps to test the model.

1 – 5 epochs: (0.966 means 96.6% accuracy.)

Epoch	GPU_mem	loss	Instances	Size	
1/25	0G	1.557	2	224: 100%	1137/1137 3.5it/s 5:21
	classes	top1_acc	top5_acc:	100%	143/143 3.8it/s 37.2s
	all	0.901	1		
Epoch	GPU_mem	loss	Instances	Size	
2/25	0G	0.3203	2	224: 100%	1137/1137 3.6it/s 5:16
	classes	top1_acc	top5_acc:	100%	143/143 3.5it/s 41.2s
	all	0.939	1		
Epoch	GPU_mem	loss	Instances	Size	
	all	0.939	1		
Epoch	GPU_mem	loss	Instances	Size	
3/25	0G	0.2264	2	224: 100%	1137/1137 3.5it/s 5:21
	classes	top1_acc	top5_acc:	100%	143/143 3.6it/s 39.9s
	all	0.958	1		
Epoch	GPU_mem	loss	Instances	Size	
4/25	0G	0.1836	2	224: 100%	1137/1137 3.6it/s 5:12
	classes	top1_acc	top5_acc:	100%	143/143 3.5it/s 40.4s
	all	0.954	1		
Epoch	GPU_mem	loss	Instances	Size	
5/25	0G	0.1627	2	224: 100%	1137/1137 3.7it/s 5:08
	classes	top1_acc	top5_acc:	100%	143/143 3.5it/s 40.7s
	all	0.966	1		

Accuracy:

- ✓ **Epoch 1: 90.1%**
- ✓ **Epoch 2: 93.9%**
- ✓ **Epoch 3: 95.8%**
- ✓ **Epoch 4: 95.4% (*a tiny dip, which is normal*)**
- ✓ **Epoch 5: 96.6%**

6 – 10 epochs: (0.975 means 97.5% accuracy.)

Epoch	GPU_mem	loss	Instances	Size	
6/25	0G	0.1515	2	224: 100%	1137/1137 3.6it/s 5:16
	classes	top1_acc	top5_acc:	100%	143/143 3.6it/s 40.0s
	all	0.967	1		
Epoch	GPU_mem	loss	Instances	Size	
7/25	0G	0.1358	2	224: 100%	1137/1137 3.6it/s 5:16
	classes	top1_acc	top5_acc:	100%	143/143 3.6it/s 39.9s
	all	0.973	1		
Epoch	GPU_mem	loss	Instances	Size	
8/25	0G	0.125	2	224: 100%	1137/1137 4.0it/s 4:47
	classes	top1_acc	top5_acc:	100%	143/143 3.8it/s 37.3s
	all	0.971	1		
Epoch	GPU_mem	loss	Instances	Size	
9/25	0G	0.1217	2	224: 100%	1137/1137 4.0it/s 4:42
	classes	top1_acc	top5_acc:	100%	143/143 3.9it/s 36.9s
	all	0.974	1		
Epoch	GPU_mem	loss	Instances	Size	
10/25	0G	0.1126	2	224: 100%	1137/1137 4.2it/s 4:32
	classes	top1_acc	top5_acc:	100%	143/143 4.0it/s 35.6s
	all	0.975	1		

Accuracy:

- ✓ **Epoch 6: 96.7%**
- ✓ **Epoch 7: 97.3%**
- ✓ **Epoch 8: 97.1%**
- ✓ **Epoch 9: 97.4%**
- ✓ **Epoch 10: 97.5%**

Data loss:

- ✓ **Epoch 6: 0.151**
- ✓ **Epoch 7: 0.135**
- ✓ **Epoch 8: 0.125**
- ✓ **Epoch 9: 0.121**
- ✓ **Epoch 10: 0.112**

11 – 15 epochs: (0.977 means 97.7% accuracy.):

Epoch	GPU_mem	loss	Instances	Size	
11/25	0G	0.1098	2	224: 100%	1137/1137 3.8it/s 5:01
	classes	top1_acc	top5_acc:	100%	143/143 3.9it/s 36.3s
	all	0.975	1		
Epoch	GPU_mem	loss	Instances	Size	
12/25	0G	0.107	2	224: 100%	1137/1137 3.3it/s 5:42
	classes	top1_acc	top5_acc:	100%	143/143 2.8it/s 50.6s
	all	0.976	1		
Epoch	GPU_mem	loss	Instances	Size	
13/25	0G	0.105	2	224: 100%	1137/1137 3.7it/s 5:10
	classes	top1_acc	top5_acc:	100%	143/143 3.9it/s 36.4s
	all	0.977	1		
Epoch	GPU_mem	loss	Instances	Size	
14/25	0G	0.09828	2	224: 100%	1137/1137 4.1it/s 4:38
	classes	top1_acc	top5_acc:	100%	143/143 4.0it/s 35.9s
	all	0.977	1		
Epoch	GPU_mem	loss	Instances	Size	
15/25	0G	0.09621	2	224: 100%	1137/1137 3.8it/s 5:02
	classes	top1_acc	top5_acc:	100%	143/143 4.0it/s 35.4s
	all	0.975	1		

Accuracy:

- ✓ Epoch 11: 97.5%
- ✓ Epoch 12: 97.6%
- ✓ Epoch 13: 97.7%
- ✓ Epoch 14: 97.7%
- ✓ Epoch 15: 97.5%

Data loss:

- ✓ Epoch 11: 0.109
- ✓ Epoch 12: 0.107
- ✓ Epoch 13: 0.105
- ✓ Epoch 14: 0.097
- ✓ Epoch 15: 0.097

16 – 20 epochs: (0.978 means 97.8% accuracy.)

Epoch	GPU_mem	loss	Instances	Size	
16/25	0G	0.09113	2	224: 100%	1137/1137 3.9it/s 4:49
	classes	top1_acc	top5_acc: 100%		143/143 4.0it/s 35.8s
	all	0.975	1		
Epoch	GPU_mem	loss	Instances	Size	
17/25	0G	0.08553	2	224: 100%	1137/1137 4.1it/s 4:36
	classes	top1_acc	top5_acc: 100%		143/143 3.9it/s 36.8s
	all	0.976	1		
Epoch	GPU_mem	loss	Instances	Size	
18/25	0G	0.08741	2	224: 100%	1137/1137 4.1it/s 4:38
	classes	top1_acc	top5_acc: 100%		143/143 4.0it/s 35.9s
	all	0.977	1		
Epoch	GPU_mem	loss	Instances	Size	
19/25	0G	0.08423	2	224: 100%	1137/1137 4.1it/s 4:35
	classes	top1_acc	top5_acc: 100%		143/143 3.9it/s 36.2s
	all	0.978	1		
Epoch	GPU_mem	loss	Instances	Size	
20/25	0G	0.08078	2	224: 100%	1137/1137 4.2it/s 4:31
	classes	top1_acc	top5_acc: 100%		143/143 3.9it/s 37.1s
	all	0.977	1		

Accuracy:

- ✓ Epoch 16: 97.5%
- ✓ Epoch 17: 97.6%
- ✓ Epoch 18: 97.7%
- ✓ Epoch 19: 97.8%
- ✓ Epoch 20: 97.7%

Data Loss:

- ✓ Epoch 16: 0.091
- ✓ Epoch 17: 0.085
- ✓ Epoch 18: 0.0741
- ✓ Epoch 19: 0.084
- ✓ Epoch 20: 0.080

20 – 25 epochs: (0.98 means 98% accuracy.)

```

Epoch  GPU_mem  loss  Instances  Size
21/25   0G      0.07617  2          224: 100% 1137/1137 3.7it/s 5:04
      classes top1_acc top5_acc: 100% 143/143 3.7it/s 38.9s
      all    0.976      1

Epoch  GPU_mem  loss  Instances  Size
22/25   0G      0.07774  2          224: 100% 1137/1137 3.5it/s 5:22
      classes top1_acc top5_acc: 100% 143/143 3.2it/s 44.1s
      all    0.978      1

Epoch  GPU_mem  loss  Instances  Size
23/25   0G      0.07476  2          224: 100% 1137/1137 3.4it/s 5:36
      classes top1_acc top5_acc: 100% 143/143 3.4it/s 42.7s
      all    0.979      1

Epoch  GPU_mem  loss  Instances  Size
24/25   0G      0.07126  2          224: 100% 1137/1137 3.5it/s 5:21
      classes top1_acc top5_acc: 100% 143/143 3.6it/s 39.9s
      all    0.979      1

Epoch  GPU_mem  loss  Instances  Size
25/25   0G      0.06996  2          224: 100% 1137/1137 3.6it/s 5:15
      classes top1_acc top5_acc: 100% 143/143 3.9it/s 37.0s
      all    0.98      1

25 epochs completed in 2.373 hours.
Optimizer stripped from C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\last.pt, 3.0MB
Optimizer stripped from C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\best.pt, 3.0MB

Validating C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\best.pt...
Ultralytics 8.4.40 Python-3.10.11 torch-2.11.0+cpu CPU (AMD Ryzen 9 6900HX with Radeon Graphics)
YOLOv8n-cls summary (fused): 30 layers, 1,465,624 parameters, 0 gradients, 3.3 GFLOPs
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train... found 18180 images in 24 classes
val: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\val... found 4556 images in 24 classes
test: None...
      classes top1_acc top5_acc: 100% 143/143 4.1it/s 34.9s
      all    0.98      1

Speed: 0.0ms preprocess, 2.2ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1

```

Accuracy:

- ✓ Epoch 21: 97.6%
- ✓ Epoch 22: 97.8%
- ✓ Epoch 23: 97.9%
- ✓ Epoch 24: 97.9%
- ✓ Epoch 25: 98%

Data Loss:

- ✓ Epoch 21: 0.076
- ✓ Epoch 22: 0.077
- ✓ Epoch 23: 0.074
- ✓ Epoch 24: 0.071
- ✓ Epoch 25: 0.069

Results saved to

C:\Users\MOHDABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1

Overall Result :

```
25 epochs completed in 2.373 hours.
Optimizer stripped from C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\last.pt, 3.0MB
Optimizer stripped from C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\best.pt, 3.0MB

Validating C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1\weights\best.pt...
Ultralytics 8.4.40 Python-3.10.11 torch-2.11.0+cpu CPU (AMD Ryzen 9 6900HX with Radeon Graphics)
YOLOv8n-cls summary (fused): 30 layers, 1,465,624 parameters, 0 gradients, 3.3 GFLOPs
train: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\train... found 18180 images in 24 classes
val: C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\yolo_dataset\val... found 4556 images in 24 classes
test: None...

classes top1_acc top5_acc 100% 143/143 4.1it/s 34.9s
all 0.98 1

Speed: 0.0ms preprocess, 2.2ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO\runs\classify\crop_doctor_run_1
```

Final Validation Accuracy

classes: all | top1_acc: 0.98 | top5_acc: 1

Hitting 98% Top-1 Accuracy is an incredible result, especially for a complex 24-class dataset

We touched on this earlier, but this is your final grade on the unseen validation dataset (the 4,556 images the model didn't use for training):

- **0.98 (98%):** Out of **100 images**, the model's absolute #1 highest-confidence guess is **correct 98 times**.

he Hardware & Architecture

Ultralytics 8.4.40 Python-3.10.11 torch-2.11.0+cpu CPU (AMD Ryzen 9 6900HX...)
YOLOv8n-cls summary (fused): 30 layers, 1,465,624 parameters, 0 gradients, 3.3 GFLOPs

- **CPU (AMD Ryzen 9...):** This confirms the model trained entirely on your processor. Training in 2.3 hours on a CPU is very impressive and speaks to the efficiency of the Ryzen chip.
- **(fused):** Before saving, YOLO optimizes itself by mathematically combining (fusing) certain layers together to make it run faster when you deploy it.
- **30 layers:** The depth of your neural network. The image passes through 30 distinct mathematical filters to figure out the disease.
- **1,465,624 parameters:** Think of parameters as tiny "dials" the AI adjusted during those 25 epochs. Your model tuned 1.4 million specific dials to perfectly differentiate between a healthy tomato leaf and one with bacterial spot.
- **3.3 GFLOPs:** Giga-Floating Point Operations per Second. This measures how computationally "heavy" the model is. 3.3 is extremely lightweight.

Testing :

```

(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image7.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Wheat - Brown Rust
Confidence: 57.28%
-----

(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - Late blight
Confidence: 98.02%
-----

(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image copy.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - Late blight
Confidence: 94.76%
-----

```

```

-----

(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image copy 2.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Corn - Common rust
Confidence: 99.69%
-----

(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image copy 3.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - healthy
Confidence: 57.59%
-----

```

```
(Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image copy 3.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - healthy
Confidence: 57.59%
-----

• (Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image_Rice.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - Late blight
Confidence: 69.45%
-----

• (Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

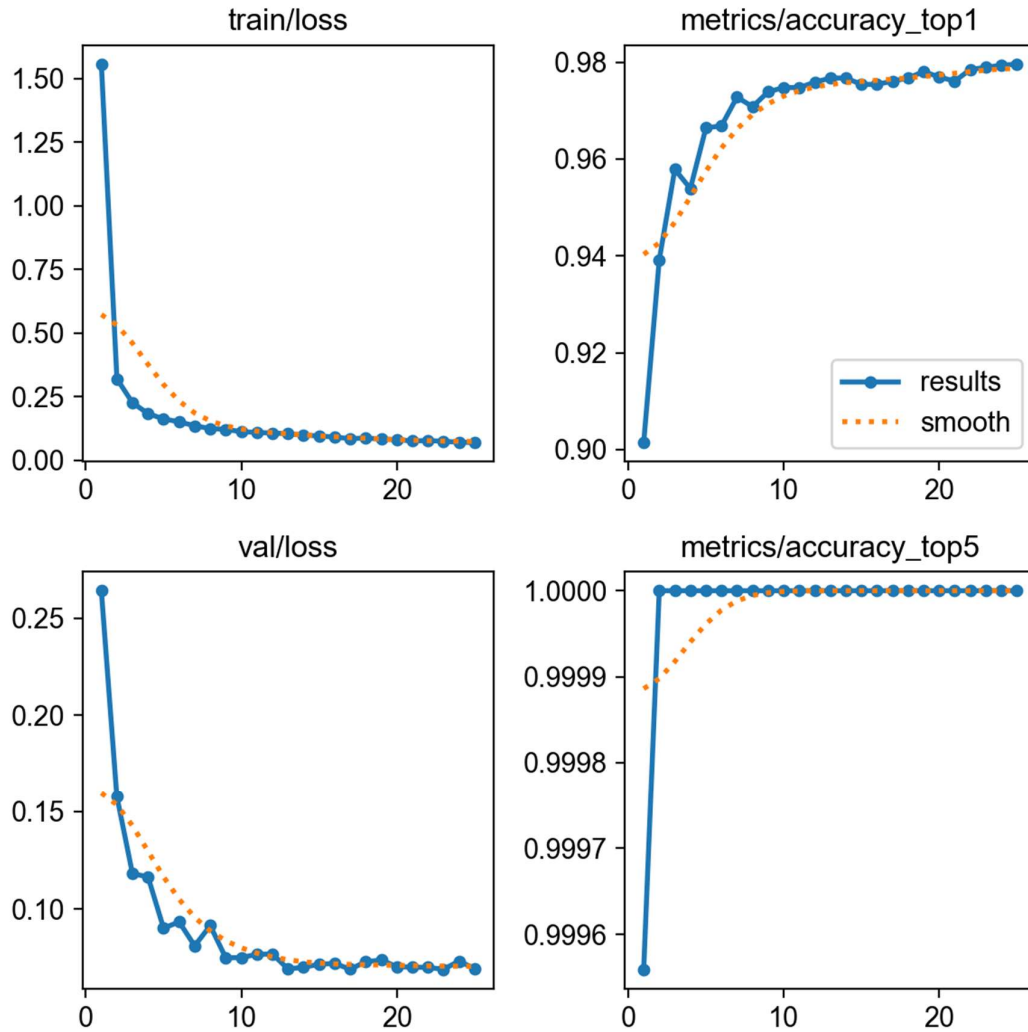
Analyzing 'image_Rice.png'...

--- AI CROP DOCTOR DIAGNOSIS ---
Result: Potato - Late blight
Confidence: 69.45%
-----

• (Cropevn) PS C:\Users\MOHD ABBAS\OneDrive\Desktop\AI_Crop_Doctor_YOLO> python 4_evaluate_model.py

Analyzing 'image_rice.png'...

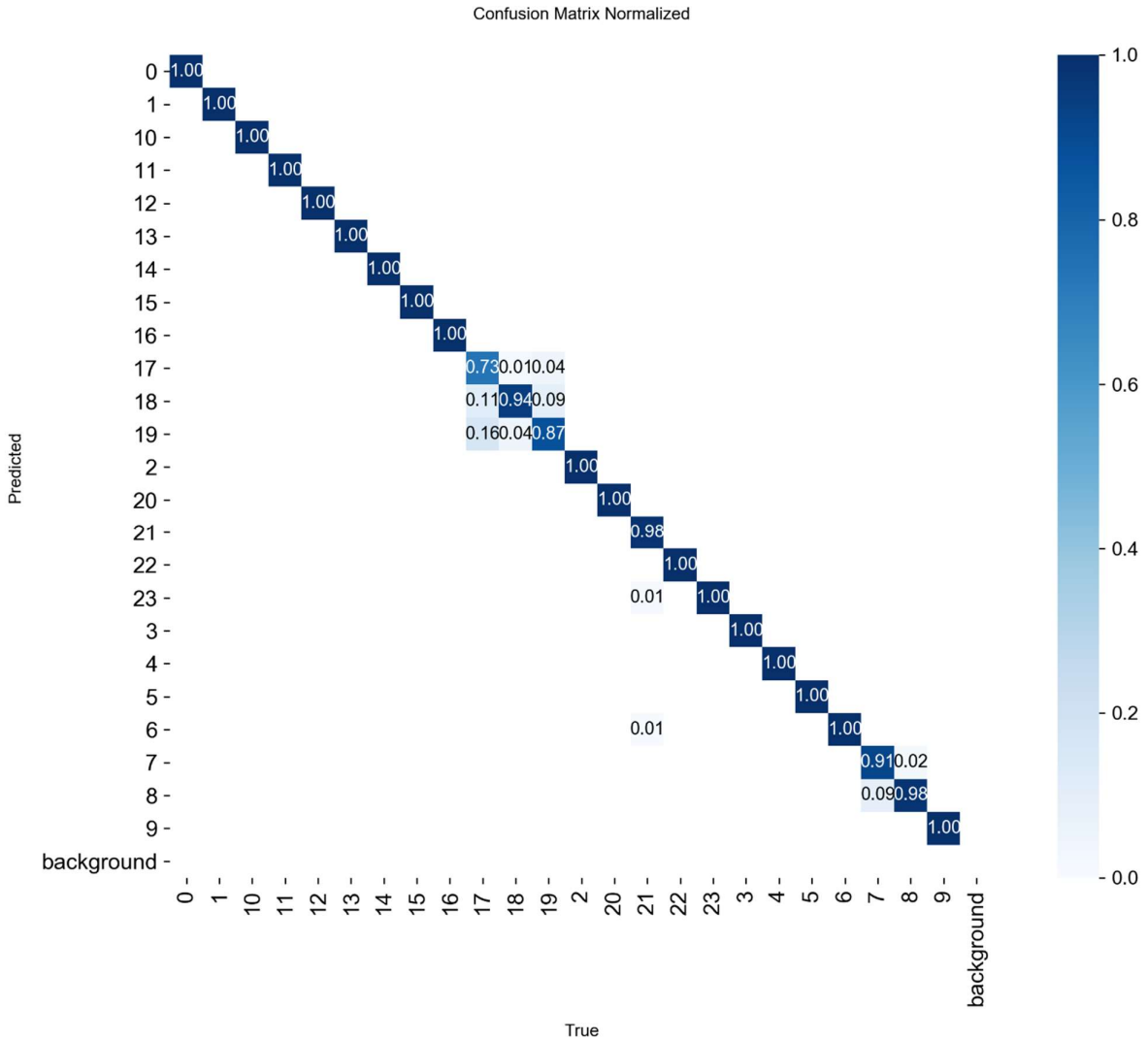
--- AI CROP DOCTOR DIAGNOSIS ---
Result: Grape - healthy
Confidence: 92.47%
-----
```



Title: Model Training Progress (Epochs 0–25)

This image contains four plots that track the model's performance as it "learns" over 25 epochs.

- **train/loss & val/loss:** Both the training and validation loss curves show a sharp initial decline that plateaus around epoch 10. The fact that the validation loss follows the training loss closely without rising suggests the model is well-fit and not significantly overfitting.
- **metrics/accuracy_top1:** This tracks how often the model's top prediction was the correct label. It reaches a very high level, approximately 98%.
- **metrics/accuracy_top5:** This tracks if the correct label was among the model's top 5 predictions. It hits 1.0 (100%) almost immediately, indicating the model is extremely confident in categorizing these classes within a small margin of error.



Title: Model Prediction Accuracy per Class

The confusion matrix provides a granular look at how the model performs on specific classes (labeled 0–23).

- **The Diagonal Line:** The deep blue diagonal represents "True Positives" (where the predicted label matches the true label). Since most values are 1.00, the model is nearly perfect for most categories.
- **Misclassifications:** There are slight "confusions" in specific clusters:
 - **Classes 17, 18, and 19:** There is some overlap here. For example, 16% of True Class 17 samples were predicted as Class 19. This usually happens when classes look very similar visually.
 - **Classes 7 and 8:** There is a minor confusion where 9% of Class 7 was predicted as Class 8.